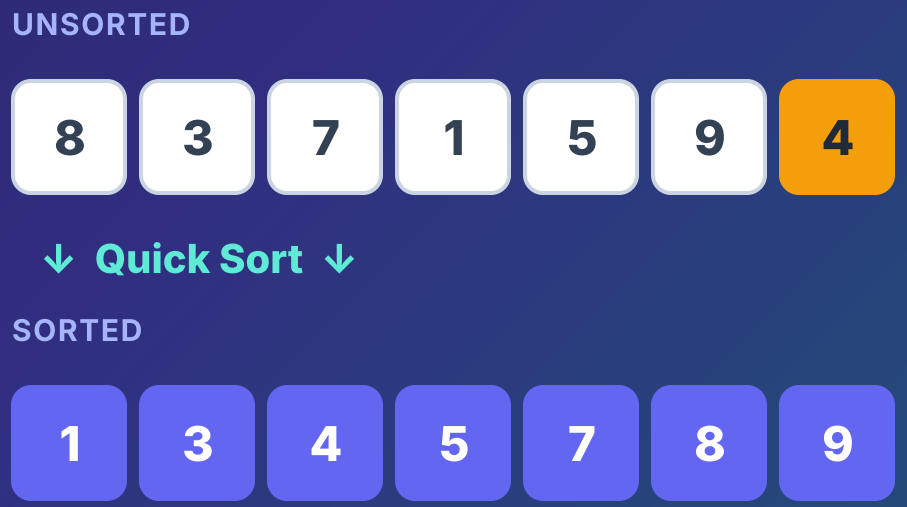
DATA STRUCTURES • ASSIGNMENT

Quick Sort

The fastest everyday sorting idea, explained step by step with diagrams and a complete C program.

Divide & Conquer | Pivot | Partition | Recursion

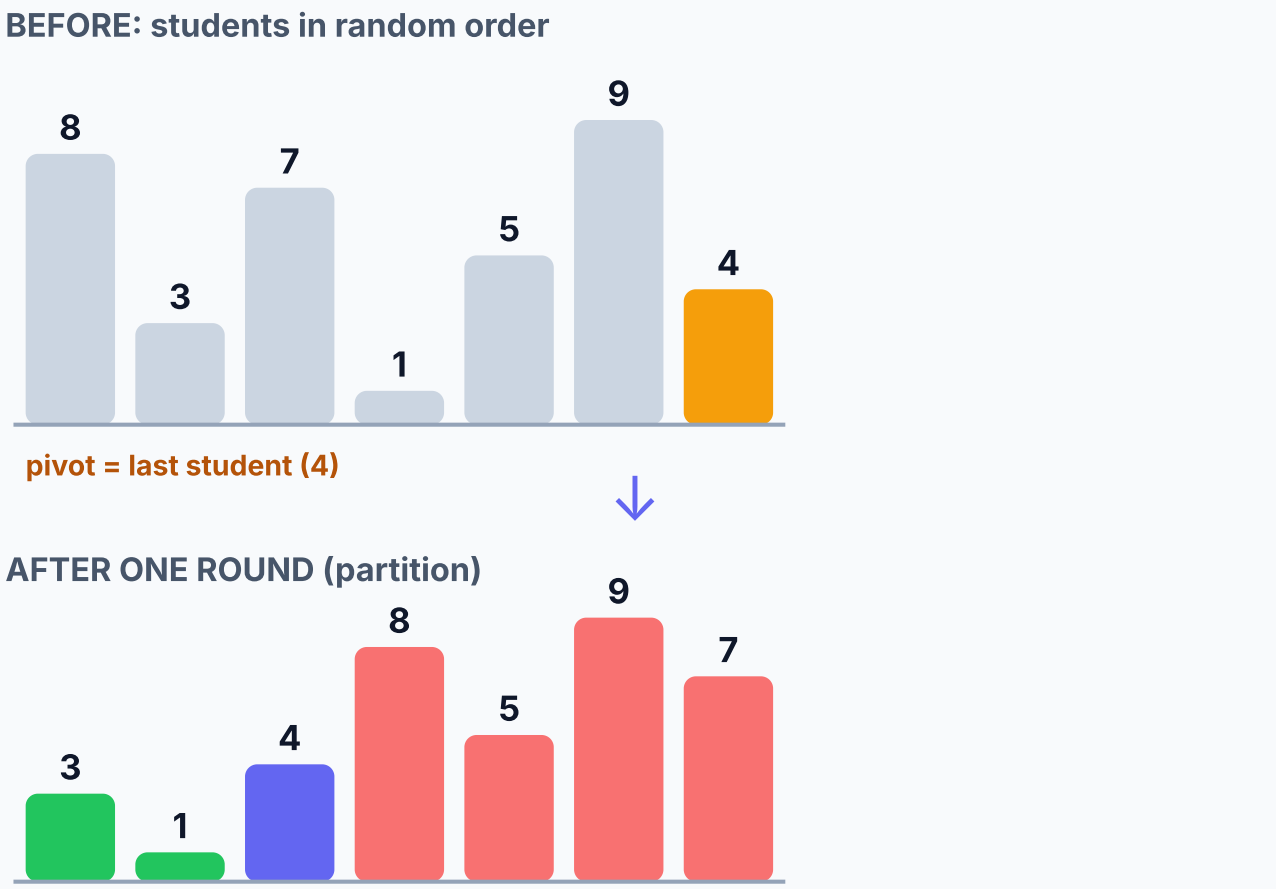


What is Quick Sort? THE IDEA

Imagine a teacher lining up students by **height**.

1. **Pick one student** — call him the **pivot**.
2. Everyone **shorter** stands on his **left**, everyone **taller** on his **right**.
3. The pivot is now at his **correct place** — he never moves again.
4. **Repeat** the same game inside the left group and the right group.

Divide & Conquer | In-place (no extra array) | Average: O(n log n)



Quick Sort in 3 Simple Steps

OVERVIEW

smaller bigger pivot final place

1 PICK a pivot

Choose one element. We always take the **last** one — easy to code.



pivot = 4

2 PARTITION

Move **smaller** numbers to the left and **bigger** numbers to the right of the pivot.



smaller | pivot | bigger

3 REPEAT

Recursion: do steps 1 & 2 again on the left part and the right part.

left part



right part



Stop when a part has **0 or 1 element** — a single number is already sorted!

Key fact: after partition the pivot sits in its **FINAL position**. It never moves again.

The Heart of It: Partition

HOW PARTITION WORKS

Partition uses **two fingers**: `i` = last element of the **smaller** area, `j` = scanner that checks one element at a time.

A SNAPSHOT WHILE PARTITION IS RUNNING (PIVOT = 4)



j sees a BIGGER number

Do nothing.
Just move **j** forward.

j sees a SMALLER number

i++, then swap `arr[i]` with `arr[j]`.
The smaller area grows by one.

j reaches the pivot

Swap pivot with `arr[i+1]`.
Pivot lands in its **final place**.

Example: One Partition

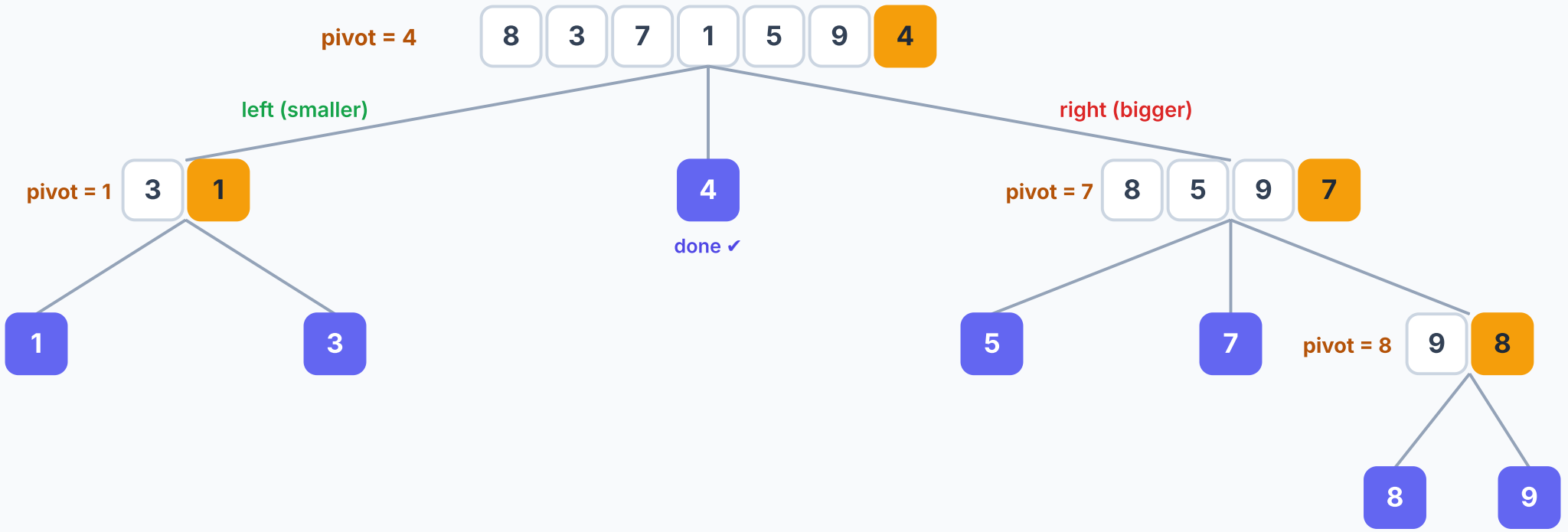
`ARRAY = 8 3 7 1 5 9 4`

smaller | bigger | pivot | final place

Start pivot = 4		`i = -1` (nothing smaller yet), `j = 0`
j = 0 → 8		8 < 4 ? **No** → bigger, skip
j = 1 → 3		3 < 4 ? **Yes** → `i=0`, swap(arr[0], arr[1])
j = 2 → 7		7 < 4 ? **No** → skip
j = 3 → 1		1 < 4 ? **Yes** → `i=1`, swap(arr[1], arr[3])
j = 4, 5 → 5, 9		Both bigger than 4 → skip, skip
Finish		swap(arr[**i+1**=2], arr[6]) → **4 is in its final place!**

Worked Example: The Whole Sort

RECURSION TREE



Collect all the finished (indigo) numbers in order:

1 3 4 5 7 8 9

✔ Sorted!

Read it like this: every box is one call of `quickSort`. Amber = pivot being used, indigo = number already in its final place.

Complete C Program `quicksort.c` FULL CODE

```c
#include <stdio.h>

/* Swap two numbers */
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

/* Partition: put the pivot at its correct place, return its index */
int partition(int arr[], int low, int high) {
    int pivot = arr[high];          // last element is the pivot
    int i = low - 1;                // i = end of the "smaller" area

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {       // found a smaller element
            i++;
            swap(&arr[i], &arr[j]); // move it into the smaller area
        }
    }
    swap(&arr[i + 1], &arr[high]);  // pivot goes to its final place
    return i + 1;                   // index of the pivot
}
```

```c
/* Quick Sort: sorts arr[low..high] */
void quickSort(int arr[], int low, int high) {
    if (low < high) {                       // at least 2 elements
        int p = partition(arr, low, high);  // pivot is now fixed at p
        quickSort(arr, low, p - 1);         // sort the left part
        quickSort(arr, p + 1, high);        // sort the right part
    }
}

/* Print all elements */
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {8, 3, 7, 1, 5, 9, 4};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Before sorting: ");
    printArray(arr, n);

    quickSort(arr, 0, n - 1);

    printf("After sorting : ");
    printArray(arr, n);
    return 0;
}
```

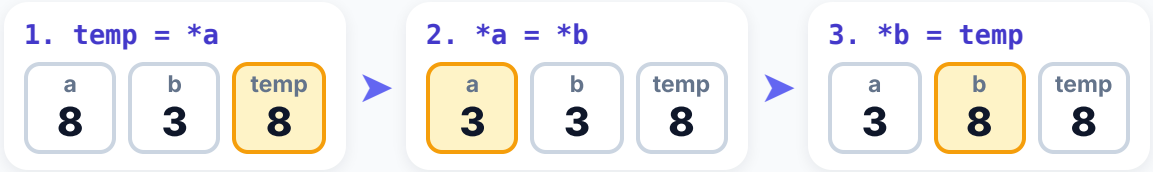
Code Explained — Part 1: swap() and main()

CODE WALK-THROUGH

swap() — exchange two values

```
4    void swap(int *a, int *b) {
5 1      int temp = *a;
6 2      *a = *b;
7 3      *b = temp;
8    }
```

Like swapping water in two glasses — you need a **third empty glass** (`temp`). We pass **addresses** (`&arr[i]`) so the function changes the **real array**, not a copy.



main() — where the program starts

```
41   int main() {
42 1     int arr[] = {8, 3, 7, 1, 5, 9, 4};
43 2     int n = sizeof(arr) / sizeof(arr[0]);
44
45 3     printf("Before sorting: ");
46       printArray(arr, n);
47
48 4     quickSort(arr, 0, n - 1);
49
50 5     printf("After sorting : ");
51       printArray(arr, n);
52       return 0;
53   }
```

1. `arr[]` — the 7 numbers we want to sort.
2. `n` = total bytes ÷ bytes of one element = 28 ÷ 4 = **7**.
3. `printArray` — helper with a loop that prints every element.
4. `quickSort(arr, 0, n-1)` — sort from index **0** to **6**.
5. Print again — now the array is sorted.

Code Explained — Part 2: partition()

THE MOST IMPORTANT FUNCTION

```
10    /* Partition: put the pivot at its correct place, return its index */
11    int partition(int arr[], int low, int high) {
12 1      int pivot = arr[high];          // last element is the pivot
13 2      int i = low - 1;                // i = end of the "smaller" area
14
15 3      for (int j = low; j < high; j++) {
16 4          if (arr[j] < pivot) {       // found a smaller element
17 4              i++;
18 4              swap(&arr[i], &arr[j]); // move it into the smaller area
19            }
20        }
21 5      swap(&arr[i + 1], &arr[high]);  // pivot goes to its final place
22 6      return i + 1;                   // index of the pivot
23    }
```

Try it with our example (pivot 4): after the loop, **i = 1** → pivot goes to index **i+1 = 2** and the function returns **2**.

Array: `8 3 7 1 5 9 4` → `3 1 4 8 5 9 7`

1 `int pivot = arr[high];`

Take the **last** element as the pivot. In our example pivot = **4**.

2 `int i = low - 1;`

i marks the end of the "smaller" area. It starts **before** the array, so the area is empty.

3 `for (int j = low; j < high; j++)`

j scans every element from the start up to (but not including) the pivot.

4 `if (arr[j] < pivot) { i++; swap(...); }`

Smaller? Grow the smaller area (`i++`) and **swap** the element into it. Bigger? Do nothing.

5 `swap(&arr[i + 1], &arr[high]);`

Put the pivot right after the smaller area — its **final sorted position**.

6 `return i + 1;`

Tell `quickSort` where the pivot is, so it can split the array there.

Code Explained — Part 3: quickSort() Recursion

DIVIDE & CONQUER

```
25    /* Quick Sort: sorts arr[low..high] */
26    void quickSort(int arr[], int low, int high) {
27 1      if (low < high) {                        // at least 2 elements
28 2          int p = partition(arr, low, high);  // pivot is now fixed at p
29 3          quickSort(arr, low, p - 1);         // sort the left part
30 4          quickSort(arr, p + 1, high);        // sort the right part
31        }
32    }
```

1. **Base case:** only run if there are **2 or more** elements. Otherwise stop — this ends the recursion.
2. **Partition** the part. The pivot is now fixed at index `p`.
3. Recursively sort the **left** part (everything before the pivot).
4. Recursively sort the **right** part (everything after the pivot).

Order of calls for `8 3 7 1 5 9 4`

#	Call	What happens	Array now
1	`quickSort(0, 6)`	pivot 4 → p = 2	`3 1 4 8 5 9 7`
2	`quickSort(0, 1)`	pivot 1 → p = 0	`1 3 4 8 5 9 7`
3	`quickSort(0, -1)`	empty → STOP	
4	`quickSort(1, 1)`	one element → STOP	
5	`quickSort(3, 6)`	pivot 7 → p = 4	`1 3 4 5 7 9 8`
6	`quickSort(3, 3)`	one element → STOP	
7	`quickSort(5, 6)`	pivot 8 → p = 5	`1 3 4 5 7 8 9`
8	`quickSort(5, 4)`	empty → STOP	
9	`quickSort(6, 6)`	one element → STOP	

Grey rows = base case reached (nothing to do). Final result: **1 3 4 5 7 8 9**

Output and Speed

RESULT

Program output

```
$ gcc quicksort.c -o quicksort
$ ./quicksort
Before sorting: 8 3 7 1 5 9 4
After sorting : 1 3 4 5 7 8 9
```

Why is it fast?

Good split

Each partition cuts the array roughly in half → about **log n** levels, each doing **n** work → **O(n log n)**

Bad split

Already-sorted array with last element as pivot → splits into 0 and n−1 → **n** levels → **O(n^2)**

Complexity at a glance

Case	Time	When
Best	**O(n log n)**	pivot splits evenly
Average	**O(n log n)**	random data
Worst	**O(n^2)**	already sorted / reversed
Extra space	O(log n)	recursion stack only
In-place?	**Yes**	no second array needed
Stable?	**No**	equal items may change order

Tip: picking a **random** or **middle** element as the pivot makes the worst case very unlikely.

Quick Recap

SUMMARY

1. **Pick** — last element is the pivot
2. **Partition** — smaller left, bigger right
3. **Place** — pivot reaches final index p
4. **Recurse** — quickSort(low, p-1) and (p+1, high)
5. **Stop** — when low ≥ high

Quick revision questions

Q1. What is a pivot?
The element used to divide the array into smaller and bigger parts.

Q2. When does the recursion stop?
When low ≥ high, i.e. the part has 0 or 1 element.

Q3. What is the worst case, and when?
$O(n^2)$ — when the array is already sorted and the last element is the pivot.

Q4. Is Quick Sort stable?
No. Equal elements can change their relative order.

Thank you! Any questions?